

逻辑回归与Softmax模型实验指导手册

实验目的

- 掌握逻辑回归模型的数学原理与实现方法
- 理解Softmax回归在多分类问题中的应用
- 学会使用梯度下降算法优化分类模型
- 掌握分类模型的评估指标与结果分析方法
- 能够应用逻辑回归和Softmax模型解决实际问题（如手写数字识别）

实验原理

1. 逻辑回归模型（二分类）

模型函数：

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

其中 $\sigma(z)$ 为Sigmoid函数

损失函数（二元交叉熵）：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

梯度更新规则：

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

2. Softmax回归模型（多分类）

模型函数：

$$P(y^{(i)} = k | x^{(i)}; \theta) = \frac{e^{\theta_k^T x^{(i)}}}{\sum_{j=1}^K e^{\theta_j^T x^{(i)}}}$$

损失函数（交叉熵）：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K 1\{y^{(i)} = k\} \log \frac{e^{\theta_k^T x^{(i)}}}{\sum_{j=1}^K e^{\theta_j^T x^{(i)}}}$$

梯度更新规则：

$$\nabla_{\theta_k} J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[x^{(i)} \left(1\{y^{(i)} = k\} - P(y^{(i)} = k | x^{(i)}; \theta) \right) \right]$$

实验步骤

1. 数据准备

使用MNIST手写数字数据集：

```
from sklearn.datasets import load_digits
```

```
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler
```

加载数据集

```
digits = load_digits()  
X, y = digits.data, digits.target
```

数据预处理

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

划分数据集

```
X_train, X_test, y_train, y_test = train_test_split(  
X_scaled, y, test_size=0.2, random_state=42  
)
```

2. 逻辑回归实现（二分类）

```
import numpy as np
```

```

class LogisticRegression:
def init(self, learning_rate=0.01, n_iters=1000):
self.lr = learning_rate
self.n_iters = n_iters
self.weights = None
self.bias = None
self.loss_history = []

def sigmoid(self, z):
return 1 / (1 + np.exp(-z))

def fit(self, X, y):
n_samples, n_features = X.shape

# 初始化参数
self.weights = np.zeros(n_features)
self.bias = 0

# 梯度下降
for _ in range(self.n_iters):
# 前向传播
linear_model = np.dot(X, self.weights) + self.bias
y_pred = self.sigmoid(linear_model)

# 计算损失
loss = -np.mean(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))
self.loss_history.append(loss)

# 反向传播
dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))
db = (1 / n_samples) * np.sum(y_pred - y)

# 更新参数
self.weights -= self.lr * dw
self.bias -= self.lr * db

def predict(self, X, threshold=0.5):
linear_model = np.dot(X, self.weights) + self.bias
y_pred = self.sigmoid(linear_model)
return (y_pred >= threshold).astype(int)

```

3. Softmax回归实现（多分类）

```
class SoftmaxRegression:
def __init__(self, learning_rate=0.01, n_iters=1000, lambda_=0.01):
    self.lr = learning_rate
    self.n_iters = n_iters
    self.lambda_ = lambda_ # L2正则化系数
    self.theta = None
    self.loss_history = []

def softmax(self, z):
    # 数值稳定性处理
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

def fit(self, X, y):
    n_samples, n_features = X.shape
    self.classes_ = np.unique(y)
    n_classes = len(self.classes_)

    # 初始化参数矩阵 (n_features x n_classes)
    self.theta = np.zeros((n_features, n_classes))

    # 将标签转换为one-hot编码
    y_onehot = np.eye(n_classes)[y]

    for i in range(self.n_iters):
        # 前向传播
        scores = np.dot(X, self.theta)
        probs = self.softmax(scores)

        # 计算损失（含L2正则化）
        data_loss = -np.mean(np.sum(y_onehot * np.log(probs + 1e-10), axis=1))
        reg_loss = 0.5 * self.lambda_ * np.sum(self.theta * self.theta)
        loss = data_loss + reg_loss
        self.loss_history.append(loss)

        # 反向传播
        gradient = (1 / n_samples) * np.dot(X.T, (probs - y_onehot))
        gradient += self.lambda_ * self.theta # L2正则化梯度
```

```

        # 更新参数
        self.theta -= self.lr * gradient

def predict(self, X):
    scores = np.dot(X, self.theta)
    probs = self.softmax(scores)
    return np.argmax(probs, axis=-1)

```

4. 模型评估

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import matplotlib.pyplot as plt
import seaborn as sns

def evaluate_model(model, X_test, y_test):
    # 预测
    y_pred = model.predict(X_test)
    # 计算准确率
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Accuracy: {accuracy:.4f}")

    # 混淆矩阵
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.title('Confusion Matrix')
    plt.show()

    # 分类报告
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    # 损失曲线
    if hasattr(model, 'loss_history'):
        plt.figure(figsize=(10, 6))

```

```
plt.plot(model.loss_history)
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.title('Training Loss History')
plt.grid(True)
plt.show()
```

5. 实验执行 (以Softmax回归为例)

创建并训练模型

```
model = SoftmaxRegression(learning_rate=0.1, n_iters=1000, lambda_=0.01)
model.fit(X_train, y_train)
```

评估模型

```
evaluate_model(model, X_test, y_test)
```

实验要求

必做任务

1. 逻辑回归实现与测试:

- 使用逻辑回归模型解决二分类问题 (如识别数字是否为"0")
- 实现并分析不同学习率 (0.001, 0.01, 0.1) 对训练过程的影响
- 绘制损失函数下降曲线和决策边界

2. Softmax回归实现与测试:

- 使用Softmax回归模型解决MNIST多分类问题
- 实现L2正则化并分析不同正则化强度 (0, 0.01, 0.1, 1) 对模型性能的影响
- 可视化混淆矩阵并分析分类错误模式

3. 模型对比分析:

- 比较逻辑回归和Softmax回归在相同任务上的表现
- 分析两种模型的训练时间、收敛速度和最终精度差异

扩展任务

1. 高级优化算法：

- 实现并比较不同优化器（SGD, Adam）的性能
- 添加学习率衰减策略并分析效果

2. 特征工程实验：

- 尝试不同的特征缩放方法（标准化、归一化）
- 使用PCA降维并分析对模型性能的影响

3. 多模型对比：

- 使用scikit-learn的LogisticRegression实现多分类
- 比较自实现模型与scikit-learn实现的性能差异
- 尝试其他分类算法（如SVM、随机森林）并与逻辑回归类模型对比

实验分析

预期结果示例

1. **损失曲线**：应呈现稳定下降趋势，最终趋于平缓
2. **混淆矩阵**：对角线值应明显高于其他位置，表明分类准确
3. **分类报告**：各类别的精确率、召回率、F1值应均衡
4. **参数分析**：
 - 学习率过大：损失震荡或发散
 - 学习率过小：收敛速度慢
 - 正则化过强：模型欠拟合
 - 正则化过弱：模型过拟合

思考题

1. 为什么逻辑回归使用交叉熵损失而不是均方误差损失？
2. Softmax函数中为什么要进行数值稳定性处理（减去最大值）？
3. 如何解释L2正则化对模型权重的影响？
4. 当类别数量极不平衡时，逻辑回归模型可能遇到什么问题？如何解决？
5. 为什么多分类问题中Softmax回归比多个二分类逻辑回归组合更优？

提示：调试时可使用小规模数据集（如鸢尾花数据集）快速验证模型正确性：

```
from sklearn.datasets import load_iris
iris = load_iris()
X, y = iris.data, iris.target
```